

Scenario

At Yettel, our customer support agents frequently need to answer questions about our various plans and services. The information is scattered across several internal documents. To improve efficiency and consistency, we want to build a prototype for an AI assistant that can answer questions based on this internal documentation.

This task requires you to build a simple, functional script and API that implements this functionality using a Retrieval-Augmented Generation (RAG) approach.

Your Mission

Build a web service (API) that takes a user's question about our services and returns an answer synthesized from a provided set of documents.

Provided Assets

You will be given a zip file containing 3 PDF documents describing Yettel products and services.

Core Requirements

1. **Data Processing:** Write a script that can load, process, and split the provided documents into smaller, manageable chunks.
2. **RAG Implementation:**
 - Use a pre-trained embedding model (e.g., from a library like sentence-transformers via Hugging Face) to create numerical representations of the document chunks.
 - Store these representations in a simple, in-memory vector store (e.g., FAISS, or even a NumPy array for simplicity).
 - Given a user query, your script should be able to find the most relevant document chunks from the vector store.
 - Use a Large Language Model (LLM) to generate a helpful answer based on the user's query and the most relevant document chunks you retrieved. You can use an accessible, open-source model or a free tier of an API service.
3. **API Service:**
 - Wrap your RAG logic in a simple web API. We recommend using **FastAPI** or **Flask**.
 - The API should have at least one endpoint, for example: POST /ask

- The endpoint should accept a JSON payload like {"query": "How much does the premium plan cost?"}.
- It should return a JSON response like {"answer": "The premium plan (Plan A) costs \$50 per month."}.
- The service should be self-contained and ready to run.

Engineering & Production Best Practices

We want to see how you structure your code and think about the problem.

- **Project Structure:** Organize your code logically. Avoid putting everything in one giant script. Think about creating separate files for different concerns (e.g., one for data loading, one for the RAG logic, one for the API).
- **Dependencies:** Include a requirements.txt file listing all necessary Python libraries.
- **Documentation (Critical):** Provide a clear README.md file. This document is as important as your code and must include:
 - Clear, step-by-step instructions on how to set up and run your application.
 - An example of how to query the API (e.g., a curl command).
 - A brief "Solution Design" section explaining your process.

What We Are Looking For (Evaluation Criteria)

We understand you have a limited time frame. Your thought process is as important as the final code. If certain feature would be too time consuming, you can write up a plan of implementation in the documentation instead. In the "Solution Design" section of your README.md, please address the following:

1. **Approach & Choices:** Briefly explain the key choices you made (e.g., which library you chose for reading PDFs, your chunking strategy, the embedding model you selected) and why you chose them.
2. **Challenges & Limitations:** What was the most challenging part of this task? What are the main weaknesses or limitations of your current prototype? (e.g., "The PDF parser struggled with tables," or "The model sometimes hallucinates if the context isn't clear.")
3. **Potential Improvements:** If you had more time, what improvements you would make? Briefly explain why. (Focus on one key area, such as improving retrieval accuracy, handling more complex documents, or making the API more robust).

Evaluation Criteria

1. **Functionality:**

- Does the core logic of the application work as expected?
- Does it successfully parse the documents and generate relevant answers?
- **A simple, working, and well-explained solution is much better than a complex, buggy one.**

2. **Code Quality & Structure:**

- Is the code clean, readable, and well-organized?
- Is the project structured in a logical and maintainable way?

3. **Problem-Solving & Explanation:**

- Is your thought process clearly explained in the README.md?
- Do you show an understanding of the task's limitations and have thoughtful ideas for improvement?

4. **Engineering Rigor:**

- Are all dependencies managed in a requirements.txt file?
- Are the instructions to run the project clear and easy to follow?
- **(Optional Bonus):** Provide a Dockerfile to containerize your application. This is not a requirement, but it is a great way to showcase additional skills.

What We are NOT Looking For

- A fancy front-end or UI.
- Fine-tuning your own LLM or using a very large, resource-heavy model.
- A complex, over-engineered solution. The goal is a simple and elegant prototype.
- Deployment to a cloud platform. Running it locally is sufficient, if that is your preference.

Deliverable

Please provide a link to a **private** Git repository (e.g., GitHub, GitLab) containing your complete solution.